

org/wiki/Ext3] explains each:

- **Journal (lowest risk):** Both metadata and file contents are written to the journal before being committed to the main filesystem. Because the journal is relatively continuous on disk, this can improve performance in some circumstances. In other cases, performance gets worse because the data must be written twice—once to the journal, and once to the main part of the filesystem.
- **Ordered (medium risk):** Only metadata is journaled; file contents are not, but it's guaranteed that file contents are written to disk before associated metadata is marked as committed in the journal. This is the default on many Linux distributions.
- **Writeback (highest risk):** Only metadata is journaled; file contents are not. The contents might be written before or after the journal is updated. As a result, files modified right before a crash can become corrupted.

These modes can be made permanent by editing the `/etc/fstab` file and specifying the journaling mode, as shown in Figure 1.

Improving journal performance

By default, the journal is located at an inode within the filesystem. To improve the performance, we should try to keep the journal on a separate device. The journaling filesystem must use the same block size as that of the data filesystem. You should always dedicate the entire partition to the journal and make sure that the external journal is always in a disk drive capable of equal or better performance.

I'm creating a normal RAID-5 device here, and then I will tune the array by using the STRIDE and CHUNK size. Finally, I'll move the journal to an external device to enhance performance.

Creating a normal RAID-5 array

Step 1 Create three partitions of 100 MB (that is, 102400 KB) each

sysfs	/sys	sysfs	defaults	0	0
proc	/proc	proc	defaults	0	0
/dev/sda5	/data	ext3	defaults	data=ordered	0
/dev/sda6	/vbox	ext3	defaults	0	0

Figure 1: Changing the journaling mode of an ext3 filesystem by editing `/etc/fstab`

[root@server1 ~]# iostat -x -d /dev/sda 1									
Linux 2.6.18-92.el5xen (server1.example.com) 09/14/2009									
Device:	rrqm/s	wrqm/s	r/s	w/s	rsec/s	wsec/s	avgrr-sz	avgwr-sz	await
sda	9.81	11.40	10.40	4.12	1461.35	123.99	109.20	0.86	4.24
Device:	rrqm/s	wrqm/s	r/s	w/s	rsec/s	wsec/s	avgrr-sz	avgwr-sz	await
sda	49.00	0.00	39.00	0.00	7816.00	0.00	200.41	0.10	2.56

Figure 2: Calculating the chunk size

and change its 'type' to `fd`.

Step 2 Use the `mdadm` command to create the array:

```
# mdadm -C /dev/md0 -l5 -n 3 /dev/sda(7,8,9)
```

Step 3 Format the array:

```
# mke2fs -b 2048 -j /dev/md0
```

Step 4 The RAID device is now ready and initiated. You can mount it on any location to use it.

Step 5 I will now capture the filesystem layout using the `dumpe2fs` command:

```
# dumpe2fs /dev/md0 > anyfilename
```

Step 6 Now stop the array and its superblock:

```
# mdadm -S /dev/zero --zero-superblock
```

I now want to reduce the disk visit count on this array by using the chunk size and stride values.

Chunk size is actually the amount of data read/written from each device in an array before moving to the next device in a round-robin manner. It is also known as 'granularity of the stripe'. It's always in KB. Chunk size is very important for RAID performance.

If the chunk size is too small, it can result in a file spreading into multiple smaller pieces thus increasing the number of drives it will use. On the other hand, if the chunk size is too big, a situation may develop in which all the IO is handled by one physical device only—a condition

Inode count:	25624576
Block count:	25599577
Reserved block count:	1279978
Free blocks:	18711564
Free inodes:	25582961
First block:	0
Block size:	4096
Fragment size:	4096
Reserved GDT blocks:	3617

Figure 3: A snippet of `dumpe2fs` output

known as 'hot spot'.

The ideal situation is to spread the IO across all the devices evenly.

To calculate the chunk size, first use the following code:

```
# iostat -x -d /dev/sda 1
```

...to get the value of `avgrr-sz` (average request size). Figure 2 displays the output on my system.

I will consider the value of 200.41 for my calculations. Just note down this value. Now divide it by 2. It's approximately 100.20.

Now divide 100.20 by the number of disk drives in your array. In my case, it was three. So I should be using the figure 100.20/3, which equals 33.4.

Round off this value to the nearest power of 2, which comes to 32. That means my chunk size is 32.

Now, before we move ahead in configuring a tuned RAID, we should also find out the stride value. Stride is the same as chunk size but uses a different value. The aim of the stride is to distribute the block bitmap across RAID member devices.

The formula to calculate the stride is: $\text{STRIDE} = \text{CHUNK SIZE} / \text{BLOCK SIZE}$

We have already retrieved the block size from the `dumpe2fs /dev/md0` command by redirecting the output to the `anyfilename` file. Now, access this file with the `less`